

Domain 2 – Account Management & Data Governance

Complete Beginner's Master Guide | COF-C03 2026

Technical + Layman Definitions | Edge Cases | Hands-On SQL | 20 Questions Per Topic | Full Answer Key

How to use this guide

1. Read the ☒ **Layman Explanation** first — build the mental model
2. Read the ☒ **Technical Explanation** — understand precise mechanics, edge cases, and exam traps
3. Run every Hands-On Query block in your Snowflake free trial (sign up: <https://signup.snowflake.com>)
4. Attempt all 20 questions per topic WITHOUT looking at answers
5. Check the Answer Key at the very bottom

Exam facts: Domain 2 = 20% of exam weight. This domain covers three tightly linked subdomains: Security Principles (2.1), Entities and Roles — RBAC (2.2), and Data Governance (2.3 + Cost Monitoring). Every question here directly impacts real-world security and compliance decisions.^{[1][2]}

SUBDOMAIN 2.1 — Snowflake Security Principles

☒ Layman's Explanation

Imagine Snowflake is a highly secure office building:

☒ **Network Policy** = The building's access gate. It decides which IP addresses (delivery trucks, employees) are even allowed to approach the front door. Everyone else gets turned away before they enter.

☒ **Authentication** = The front door lock. Even if you're allowed on the property, you still need the right credentials to unlock the door. Options range from a simple password (easy but weak) to a fingerprint + PIN combo (MFA), a company badge reader (SSO/SAML), or a cryptographic key pair (military-grade lock).

☒ **Authentication Policies** = The building's security rulebook. "All HR staff MUST use two locks to enter. IT staff may use either the badge reader OR the key." These are written rules enforced automatically for every entry attempt.

☒ Technical Explanation

Network Policies[^3]

What they are: Account-level (or user-level) rules that restrict Snowflake access by IP address.

How they work:

- Define an `ALLOWED_IP_LIST` (whitelist) — only these IPs can connect
- Define a `BLOCKED_IP_LIST` (blacklist) — these IPs are always denied, even if in the allowed list
- Can be applied at the **account level** (affects all users) or **user level** (overrides account policy for that specific user)
- User-level policy takes precedence over account-level policy

Key behaviors and edge cases:

- A user with no user-level network policy is governed by the account-level policy
- If NO account-level network policy is set, ALL IPs can connect (no restriction)
- `BLOCKED_IP_LIST` takes precedence over `ALLOWED_IP_LIST` — if an IP is in both lists, it is `BLOCKED`
- Network policies restrict connection attempts — not query execution (once connected, the policy is already satisfied)
- Network policies are NOT evaluated for internal Snowflake-to-Snowflake traffic (e.g., replication)
- Creating a network policy does NOT automatically activate it — you must **APPLY** it to an account or user

```
-- Syntax:
CREATE NETWORK POLICY policy_name
    ALLOWED_IP_LIST = ('192.168.1.0/24', '10.0.0.5')
    BLOCKED_IP_LIST = ('192.168.1.99');

-- Apply to account:
ALTER ACCOUNT SET NETWORK_POLICY = policy_name;

-- Apply to specific user:
ALTER USER myuser SET NETWORK_POLICY = policy_name;

-- Remove from user (falls back to account-level policy):
ALTER USER myuser UNSET NETWORK_POLICY;
```

Authentication Methods^{[4][5][6][7]}

Snowflake supports multiple authentication methods. Understanding each one is critical for the exam.

Method	Description	Best For
Password	Standard username + password. Being deprecated for human users.	Legacy; being phased out
MFA (Multi-Factor Authentication)	Password + second factor (Duo, authenticator app TOTP, passkey). Three supported methods: passkey, TOTP app, Duo.	All human users — now mandatory
Key Pair Authentication	RSA or ECDSA public/private key pair. Public key stored in Snowflake user object. Private key used by client to sign connection. No password transmitted.	Service accounts, automation, CI/CD
OAuth	Token-based delegated authorization. User grants a third-party app (Tableau, dbt) access to Snowflake without sharing credentials. Tokens are time-limited.	BI tools, application integrations
SAML 2.0 / SSO	Federated authentication via an external Identity Provider (IdP) like Okta, Azure AD, Ping. User's corporate login authenticates to Snowflake.	Enterprise SSO environments
Federated Authentication	Broader term encompassing SAML 2.0 + any IdP-based login	Enterprise identity management
Programmatic Access Tokens (PAT)	New token type for programmatic access — long-lived tokens for service accounts. Alternative to key-pair.	Service accounts
Workload Identity Federation (WIF)	Cloud-native identity (AWS IAM Role, Azure Managed Identity, GCP Service Account) used to authenticate to Snowflake without any stored credentials.	Cloud-native service accounts

2026 enforcement timeline (important for exam):^{[5][7]}

- MFA is now **mandatory** for all human users logging into Snowsight
- Key pair / OAuth / PAT / WIF are the required methods for **service accounts**
- Password-only logins for human users: **deprecated**
- The user attribute TYPE matters: PERSON (human), SERVICE (automated), LEGACY_SERVICE (being deprecated)

Multi-Factor Authentication (MFA) Deep Dive^[^6]

Supported MFA methods:

1. Passkey — hardware or software passkey (FIDO2 standard, phishing-resistant)
2. TOTP (Time-Based One-Time Password) — 6-digit codes from an authenticator app (Google Authenticator, Authy)
3. Duo — push notification or passcode via Duo Security app

MFA with non-browser clients:

- ODBC driver: use `passcode=de>` OR `passcodeInPassword=on` in connection parameters
- By default, Duo Push is used for ODBC/JDBC when enrolled in MFA

Authentication Policies:[^8]

- A Snowflake object (`CREATE AUTHENTICATION POLICY`) that defines WHICH authentication methods are allowed, MFA requirements, and which IdP integrations are permitted
- Applied to accounts or users
- Key parameters: `AUTHENTICATION_METHODS`, `MFA_ENROLLMENT` (REQUIRED or OPTIONAL), `MFA_POLICY`, `SECURITY_INTEGRATIONS`
- User-level authentication policy overrides account-level

Key Pair Authentication Deep Dive[^5]

How it works:

1. User generates RSA key pair (minimum 2048-bit; recommended 4096-bit)
2. Public key is uploaded to the Snowflake user object: `ALTER USER myuser SET RSA_PUBLIC_KEY = '...'`
3. Private key stays on the client — never sent to Snowflake
4. Client uses private key to sign a JWT (JSON Web Token) assertion
5. Snowflake verifies the signature using the stored public key
6. If valid → session granted; no password ever transmitted

Edge cases:

- A user can have TWO public keys simultaneously (`RSA_PUBLIC_KEY` and `RSA_PUBLIC_KEY_2`) — enables key rotation without downtime
- Private keys can (and should) be passphrase-protected

Single Sign-On (SSO) and SAML 2.0[^4][8]

What it is: An external Identity Provider (IdP) handles authentication. When a user logs into their corporate system (e.g., Okta), a SAML assertion token is generated and presented to Snowflake. Snowflake trusts the IdP's assertion without requiring its own password check.

Setup requirement: A Security Integration object (`CREATE SECURITY INTEGRATION TYPE=SAML2`) must be created in Snowflake to point to the IdP metadata.

Edge case: By default, Snowflake does NOT require MFA for SSO users — it relies on the IdP to enforce MFA. You can force SSO users to also pass Snowflake MFA using an Authentication Policy with `ENFORCE_MFA_ON_EXTERNAL_AUTHENTICATION='ALL'`.[^6]

Encryption in Snowflake^[9][10][^11]

Layer	Method	Details
Data in transit	TLS 1.2+	All client-to-Snowflake traffic is encrypted in transit
Data at rest	AES-256	All stored data encrypted always — no opt-out
Key management	Hierarchical key model	Account Master Key → Table Master Keys → File Keys → Data
Key rotation	Automatic every 30 days	Snowflake rotates encryption keys automatically
BYOK / Tri-Secret Secure	Customer-Managed Key (CMK) + Snowflake key = Composite Master Key	Business Critical edition only

Tri-Secret Secure:^[12][13][^9]

- Requires Business Critical edition
- Uses a **dual-key model**: Snowflake holds one key, you hold a Customer-Managed Key (CMK) in your cloud provider's KMS (AWS KMS, Azure Key Vault, GCP Cloud KMS)
- Both keys are combined to form a **Composite Master Key** — neither key alone can decrypt your data
- If you revoke your CMK → Snowflake CANNOT decrypt your data (even Snowflake can't access it)
- Critical exam trap: Tri-Secret Secure is NOT just "bring your own key" — it's a **triple-layer protection**: data encryption + Snowflake key + your key

☒ Hands-On Queries: Security Principles

```
-- =====
-- PART 1: Network Policies
-- =====

USE ROLE ACCOUNTADMIN;

-- Create a network policy
CREATE OR REPLACE NETWORK POLICY office_only_policy
  ALLOWED_IP_LIST = ('203.0.113.0/24', '198.51.100.50')
  BLOCKED_IP_LIST = ('203.0.113.99')
  COMMENT = 'Allow corporate office IPs only; block rogue IP';

-- View the policy definition
SHOW NETWORK POLICIES;
DESC NETWORK POLICY office_only_policy;

-- Apply to the account (blocks all IPs not in allowed list for everyone)
-- WARNING: Only run this if you are sure your current IP is in ALLOWED_IP_LIST!
-- ALTER ACCOUNT SET NETWORK_POLICY = office_only_policy;
```

```

-- Apply to a specific user only
CREATE USER IF NOT EXISTS test_user_security
    PASSWORD = 'TempPass@123'
    DEFAULT_ROLE = PUBLIC;

ALTER USER test_user_security SET NETWORK_POLICY = office_only_policy;

-- Verify the user-level network policy
DESC USER test_user_security;
-- Shows: network_policy = OFFICE_ONLY_POLICY

-- Remove user-level policy (falls back to account-level)
ALTER USER test_user_security UNSET NETWORK_POLICY;

-- Cleanup
DROP NETWORK POLICY IF EXISTS office_only_policy;
DROP USER IF EXISTS test_user_security;

-- =====
-- PART 2: User Types and Authentication
-- =====

USE ROLE USERADMIN;

-- Create a HUMAN user (TYPE = PERSON) - requires MFA
CREATE OR REPLACE USER analyst_john
    PASSWORD      = 'ChangeMe@456'
    LOGIN_NAME    = 'john.analyst@company.com'
    DISPLAY_NAME  = 'John Analyst'
    EMAIL         = 'john@company.com'
    TYPE          = PERSON          -- Human user → MFA required
    DEFAULT_ROLE  = PUBLIC
    COMMENT       = 'Human user - MFA will be enforced';

-- Create a SERVICE user for automation (no MFA, use key-pair)
CREATE OR REPLACE USER etl_service_account
    LOGIN_NAME    = 'ETL_SERVICE'
    TYPE          = SERVICE         -- Machine user → key-pair/OAuth required
    DEFAULT_ROLE  = PUBLIC
    COMMENT       = 'ETL automation service account';

-- Set RSA public key for key-pair authentication (placeholder key)
-- In real usage, replace with your actual generated public key:
-- ALTER USER etl_service_account SET RSA_PUBLIC_KEY = 'MIIBIjANBgkq...';

-- A user can have TWO keys for rotation (RSA_PUBLIC_KEY_2)
-- ALTER USER etl_service_account SET RSA_PUBLIC_KEY_2 = 'MIIBIjANBgkq...';
-- After switching clients to use key 2, remove key 1:
-- ALTER USER etl_service_account UNSET RSA_PUBLIC_KEY;

-- Check user details
DESC USER analyst_john;
-- Look for: type, mfa_enrollment, last_success_login

SHOW USERS;
-- See all users with their types, default roles

```

```

-- =====
-- PART 3: Authentication Policies
-- =====

USE ROLE ACCOUNTADMIN;

-- Create an Authentication Policy requiring MFA for all password logins
CREATE OR REPLACE AUTHENTICATION POLICY strong_auth_policy
    AUTHENTICATION_METHODS = ('PASSWORD', 'SAML', 'KEYPAIR') -- allowed methods
    MFA_ENROLLMENT          = 'REQUIRED'                    -- MFA is mandatory
    COMMENT                  = 'Enforce MFA for all human users';

-- Apply to entire account
ALTER ACCOUNT SET AUTHENTICATION_POLICY = strong_auth_policy;

-- Create a stricter policy for service accounts (key-pair only)
CREATE OR REPLACE AUTHENTICATION POLICY service_auth_policy
    AUTHENTICATION_METHODS = ('KEYPAIR') -- ONLY key-pair allowed
    MFA_ENROLLMENT          = 'OPTIONAL' -- MFA not applicable for service accounts
    COMMENT                  = 'Service accounts must use key-pair only';

-- Apply to the service account user
ALTER USER etl_service_account SET AUTHENTICATION_POLICY = service_auth_policy;

-- View authentication policies
SHOW AUTHENTICATION POLICIES;

-- Remove account-level policy
ALTER ACCOUNT UNSET AUTHENTICATION_POLICY;

-- =====
-- PART 4: Check encryption and security settings
-- =====

USE ROLE ACCOUNTADMIN;

-- View account-level security parameters
SHOW PARAMETERS LIKE '%REQUIRE%' IN ACCOUNT;
SHOW PARAMETERS LIKE '%PASSWORD%' IN ACCOUNT;

-- Check key rotation info (if Tri-Secret Secure is enabled)
-- SELECT SYSTEM$GET_CMK_INFO(); -- Only works with Business Critical + Tri-Secret S

-- View recent login history (who logged in, how, from where)
SELECT
    USER_NAME,
    EVENT_TIMESTAMP,
    IS_SUCCESS,
    FIRST_AUTHENTICATION_FACTOR,
    SECOND_AUTHENTICATION_FACTOR,
    CLIENT_IP,
    REPORTED_CLIENT_TYPE
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE EVENT_TIMESTAMP >= DATEADD('hour', -24, CURRENT_TIMESTAMP())
ORDER BY EVENT_TIMESTAMP DESC
LIMIT 20;

-- Shows MFA usage: FIRST_FACTOR = PASSWORD, SECOND_FACTOR = DUO

```

```
-- Audit failed logins
SELECT
    USER_NAME,
    EVENT_TIMESTAMP,
    ERROR_MESSAGE,
    CLIENT_IP
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE IS_SUCCESS = 'NO'
    AND EVENT_TIMESTAMP >= DATEADD('day', -7, CURRENT_TIMESTAMP())
ORDER BY EVENT_TIMESTAMP DESC;

-- Clean up
DROP USER IF EXISTS analyst_john;
DROP USER IF EXISTS etl_service_account;
DROP AUTHENTICATION POLICY IF EXISTS strong_auth_policy;
DROP AUTHENTICATION POLICY IF EXISTS service_auth_policy;
```

☒ Topic 2.1 — 20 Practice Questions

Q1. What does a Snowflake Network Policy control?

- A) Which SQL operations (SELECT, INSERT, DELETE) a user can run
- B) Which IP addresses are allowed or blocked from connecting to Snowflake
- C) The maximum number of concurrent connections per user
- D) The bandwidth limit for data transfers between Snowflake and clients

Q2. A Network Policy defines both `ALLOWED_IP_LIST = ('10.0.0.0/8')` and `BLOCKED_IP_LIST = ('10.0.0.50')`. A user connecting from IP 10.0.0.50 attempts to log in. What happens?

- A) The connection succeeds — `ALLOWED_IP_LIST` takes precedence
- B) The connection is blocked — `BLOCKED_IP_LIST` takes precedence over `ALLOWED_IP_LIST`
- C) The user gets a warning but can proceed after acknowledging
- D) The system randomly enforces one list over the other

Q3. Which authentication method is MOST appropriate for a fully automated ETL pipeline service account?

- A) Password authentication — easiest to configure
- B) MFA with Duo Push — most secure
- C) Key Pair (RSA) authentication — no password transmitted; public key stored in Snowflake
- D) SAML SSO — best for automated service accounts

Q4. What does Tri-Secret Secure require in terms of Snowflake edition?

- A) Standard edition

- B) Enterprise edition
- C) Business Critical edition
- D) Virtual Private Snowflake edition

Q5. What are the three supported MFA methods in Snowflake? (*Select ALL that apply*)

- A) Passkey (FIDO2)
- B) TOTP via authenticator app (Google Authenticator, Authy)
- C) Hardware USB security key (FIDO U2F/FIDO2 stored in passkey)
- D) Duo (push notification or passcode)
- E) SMS text message code

Q6. In Tri-Secret Secure, if a customer revokes their Customer-Managed Key (CMK), what happens?

- A) Snowflake automatically generates a new replacement key within 24 hours
- B) Only new data is unreadable; existing data remains accessible
- C) Snowflake cannot decrypt the data — even Snowflake employees lose access to the account's data
- D) The account switches to standard AES-256 encryption using only Snowflake's key

Q7. A user-level Network Policy and an account-level Network Policy both exist. A user has a user-level policy assigned. Which policy governs that user's connections?

- A) Account-level policy — it always takes precedence
- B) Both policies are evaluated and the more restrictive one applies
- C) User-level policy — it overrides the account-level policy for that specific user
- D) The most recently created policy takes precedence

Q8. What happens when you CREATE a Network Policy in Snowflake but do NOT apply it to any account or user?

- A) It is automatically applied to the entire account
- B) It has no effect — creating a policy without applying it does not restrict access
- C) It applies to all newly created users but not existing ones
- D) It generates an error — policies must be applied upon creation

Q9. Key pair authentication allows a user to have TWO RSA public keys simultaneously. What is the primary purpose of this capability?

- A) To allow two users to share the same service account credentials
- B) To support key rotation without downtime — clients can migrate from key 1 to key 2 without interrupting service

- C) To double the security by requiring both keys to authenticate
- D) To support both RSA-2048 and RSA-4096 key sizes simultaneously

Q10. Which Snowflake feature requires creating a SECURITY INTEGRATION of type SAML2 before it can function?

- A) Multi-Factor Authentication (MFA)
- B) Key Pair Authentication
- C) SAML 2.0 / Single Sign-On (SSO) with an external Identity Provider
- D) Network Policy enforcement

Q11. What does an Authentication Policy object define? *(Select ALL that apply)*

- A) Which IP addresses can connect to Snowflake
- B) Which authentication methods are permitted (PASSWORD, SAML, KEYPAIR, etc.)
- C) Whether MFA enrollment is REQUIRED or OPTIONAL
- D) Which SAML security integrations can be used for login
- E) Which databases a user is allowed to access

Q12. Snowflake encrypts all data at rest using which algorithm?

- A) AES-128
- B) AES-256
- C) RSA-4096
- D) SHA-256

Q13. How often does Snowflake automatically rotate encryption keys by default?

- A) Every 7 days
- B) Every 30 days
- C) Every 90 days
- D) Once per year

Q14. A user is enrolled in MFA and uses an ODBC driver to connect. By default, which MFA method is triggered?

- A) TOTP code entry
- B) Passkey prompt
- C) Duo Push notification
- D) SMS text message

Q15. What USER TYPE should be assigned to an automated Python ETL script that connects to Snowflake?

- A) TYPE = PERSON
- B) TYPE = HUMAN
- C) TYPE = SERVICE
- D) TYPE = AUTOMATION

Q16. Which statement about Snowflake data encryption is TRUE? *(Select ALL that apply)*

- A) Data in transit is encrypted using TLS 1.2 or higher
- B) Data at rest is encrypted with AES-256 — always, automatically, with no customer opt-in required
- C) Customers must enable encryption manually per table in Enterprise edition
- D) Snowflake uses a hierarchical key model: Account Master Key → Table Master Keys → File Keys
- E) Tri-Secret Secure uses a composite master key combining Snowflake's key and a customer-managed key

Q17. An account-level Authentication Policy restricts login to KEYPAIR only. A specific user also has a user-level Authentication Policy allowing both PASSWORD and KEYPAIR. Which policy governs that user?

- A) Account-level policy — it governs all users regardless of user-level settings
- B) User-level policy — it overrides the account-level policy for that specific user
- C) The most restrictive policy applies — user can only use KEYPAIR
- D) Both policies apply — user must satisfy both simultaneously

Q18. What is the difference between RSA_PUBLIC_KEY and RSA_PUBLIC_KEY_2 on a Snowflake user?

- A) RSA_PUBLIC_KEY is for read operations; RSA_PUBLIC_KEY_2 is for write operations
- B) They are identical — Snowflake assigns the second key automatically for redundancy
- C) They are two independent public key slots — both can be active simultaneously to support seamless key rotation
- D) RSA_PUBLIC_KEY_2 is only used when the primary key expires

Q19. Which of the following are TRUE about Snowflake Network Policies? *(Select ALL that apply)*

- A) They can be applied at the account level or the user level
- B) They can be applied at the schema level to restrict table access by IP
- C) User-level policies override account-level policies for that user
- D) A policy must be applied to an account or user to take effect — creating it alone does nothing
- E) The BLOCKED_IP_LIST takes precedence over ALLOWED_IP_LIST

Q20. OAuth in Snowflake is primarily used for:

- A) Automating credential rotation for service accounts
- B) Allowing a third-party tool (e.g., Tableau, dbt) to access Snowflake on behalf of a user, using tokens instead of the user's actual credentials
- C) Encrypting data in transit between the client and Snowflake
- D) Enforcing row-level security policies based on user identity

SUBDOMAIN 2.2 — Snowflake RBAC: Roles, Entities, and Access Control

☒ Layman's Explanation

RBAC (Role-Based Access Control) is Snowflake's system for deciding **who can do what, to which objects**.

Think of it like an office building with key cards:

- **Users** = the people who want to enter
- **Roles** = the different types of key cards (visitor card, employee card, manager card, security card)
- **Privileges** = what each key card can unlock (Visitor card unlocks lobby only; Manager card unlocks all offices)
- **Role hierarchy** = Manager card inherits everything an employee card unlocks, PLUS more

The golden Snowflake rule: **Users don't get privileges directly. Users get ROLES. Roles get PRIVILEGES. Roles can inherit other roles.** This is called the DAC + RBAC model.

Functional vs. Access Roles:

- Think of **Access Roles** as "I can READ this specific dataset"
- Think of **Functional Roles** as "I am a Data Engineer" — these inherit multiple Access Roles

☒ Technical Explanation

Access Control Framework: RBAC + DAC^{[14][15][^16]}

Snowflake combines two models:

- **RBAC (Role-Based Access Control):** Privileges are granted to roles, roles are granted to users
- **DAC (Discretionary Access Control):** Every object has an OWNER (the role that created it). Owners can grant access to others

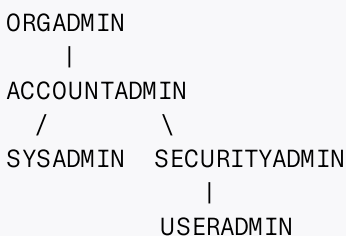
The key implication of DAC: The role that creates an object automatically becomes its owner and inherits the OWNERSHIP privilege — which includes the right to grant/revoke access to that object.

System-Defined Roles (Built-In Roles)^{[17][15][3][14]}

These are pre-created by Snowflake at account provisioning. You cannot drop them.

Role	Description	Key Privileges
ORGADMIN	Organization-level administrator. Manages accounts within an Organization.	Create/manage accounts at org level; view org-level billing
ACCOUNTADMIN	Top-level account role. Encapsulates SYSADMIN + SECURITYADMIN. God-mode.	Everything in the account — all privileges
SECURITYADMIN	Manages security. Inherits USERADMIN.	MANAGE GRANTS (can grant/revoke ANY privilege on ANY object); create/manage roles and users
USERADMIN	Manages users and roles only.	CREATE USER, CREATE ROLE — does NOT inherit other system roles by default
SYSADMIN	Manages data objects.	Create warehouses, databases, schemas, tables — all database objects
PUBLIC	Automatically granted to every user.	Minimal; can be used for objects you want all users to see

Role Hierarchy (parent → child, arrows = "inherits from"):



- ACCOUNTADMIN inherits ALL privileges from both SYSADMIN and SECURITYADMIN
- SECURITYADMIN inherits all privileges from USERADMIN
- SYSADMIN does NOT inherit SECURITYADMIN or USERADMIN
- PUBLIC is a flat role — no parent roles

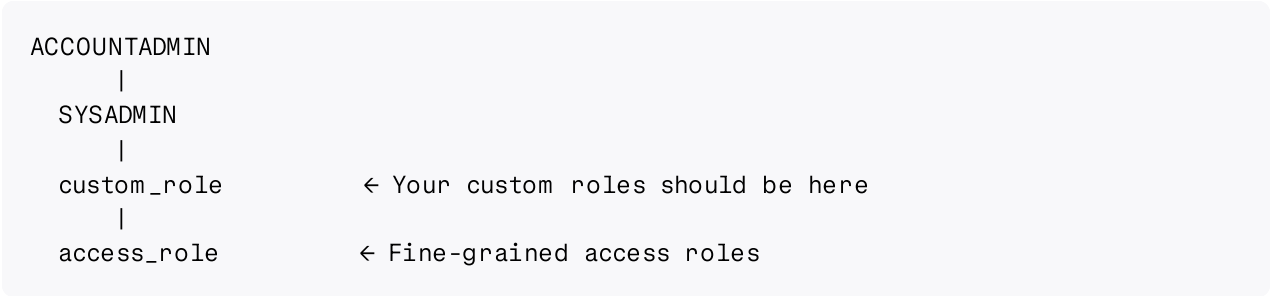
Critical exam traps:

Trap	Correct Answer
"Who should create databases?"	SYSADMIN (not ACCOUNTADMIN — best practice is to use least-privilege)
"Which role creates users?"	USERADMIN (or SECURITYADMIN — it inherits USERADMIN)

Trap	Correct Answer
"Which role can revoke ANY privilege?"	SECURITYADMIN (has MANAGE GRANTS)
"Is SYSADMIN a parent of SECURITYADMIN?"	NO — SYSADMIN does NOT inherit SECURITYADMIN
"What is ORGADMIN for?"	Managing the ORGANIZATION (multiple accounts), not a single account
"Can USERADMIN create databases?"	NO — only create users and roles

Creating and Managing Custom Roles^{[15][3][^17]}

Best practice: Custom roles should ultimately roll up to SYSADMIN (by granting the custom role TO SYSADMIN). This ensures SYSADMIN (and therefore ACCOUNTADMIN) retains visibility into all objects.



Principle of Least Privilege: Users and roles should only have the minimum privileges required to do their jobs. Never give everyone ACCOUNTADMIN.

Privileges: Types and Granting^{[18][3][^14]}

Privilege levels by object type:

Object Type	Available Privileges
Database	USAGE, CREATE SCHEMA, MODIFY, MONITOR, OWNERSHIP
Schema	USAGE, CREATE TABLE, CREATE VIEW, CREATE STAGE, MODIFY, MONITOR, OWNERSHIP
Table	SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, OWNERSHIP
View	SELECT, REFERENCES, OWNERSHIP
Warehouse	USAGE, MODIFY, MONITOR, OPERATE, OWNERSHIP
Stage	READ, WRITE, USAGE, OWNERSHIP
Masking Policy	APPLY, OWNERSHIP
Row Access Policy	APPLY, OWNERSHIP

Granting Syntax:

```
GRANT privilege ON object_type object_name TO ROLE role_name;  
REVOKE privilege ON object_type object_name FROM ROLE role_name;
```

Special cases:

- To query a table, a role needs: `USAGE` on the database + `USAGE` on the schema + `SELECT` on the table
- Missing any one of these three = access denied (all three required!)

GRANT OPTION and WITH GRANT OPTION[^14]

```
GRANT SELECT ON TABLE my_table TO ROLE analyst WITH GRANT OPTION;
```

The `WITH GRANT OPTION` allows the grantee role to **re-grant** the same privilege to other roles. Without it, the role can `USE` the privilege but **NOT** delegate it further.

FUTURE GRANTS[^3]

A mechanism to automatically grant privileges to **objects that don't exist yet** in a schema.

```
GRANT SELECT ON FUTURE TABLES IN SCHEMA mydb.myschema TO ROLE analyst;
```

After this command, any **NEW** table created in `mydb.myschema` automatically gets `SELECT` granted to `analyst`. Existing tables are unaffected — only future ones.

Edge case: `FUTURE GRANTS` don't retroactively apply to existing objects. Run a `GRANT` on existing objects separately.

Functional Roles vs. Access Roles (Recommended Pattern)[^17]

Functional Roles = "Job title" roles. Represent a persona. Examples: `data_engineer`, `data_scientist`, `analyst`

Access Roles = "Permission" roles. Represent specific privileges on specific objects. Examples: `sales_db_read`, `hr_table_write`

Best practice: Grant Access Roles to Functional Roles. Grant Functional Roles to users.

```
User John → gets role: data_engineer (functional)  
data_engineer role → has: sales_db_read, hr_table_write (access roles)
```

Secondary Roles[^19]

A user can have a DEFAULT ROLE (the primary active role when a session starts) but also activate SECONDARY ROLES using:

```
USE SECONDARY ROLES ALL;      -- Activates all roles granted to the user simultaneously
USE SECONDARY ROLES NONE;    -- Returns to primary role only
```

Edge case: Secondary Roles allow combining privileges from multiple roles in one session. This is useful when a user needs privileges from two different roles simultaneously without creating a new combined role.

Account Identifier[^19]

Every Snowflake account has a unique identifier used in connection URLs:

- Format: <org_name>-<account_name> (new format, preferred)
- Legacy format: <account_locator>.loud_region_id>.loud>
- Example URL: <https://myorg-myaccount.snowflakecomputing.com>

Edge case: Within ACCOUNT_USAGE and metadata functions, CURRENT_ACCOUNT() returns the account locator, while CURRENT_ORGANIZATION_NAME() returns the org name.

☒ Hands-On Queries: RBAC in Action

```
-- =====
-- PART 1: Explore system-defined roles
-- =====

USE ROLE ACCOUNTADMIN;

-- See all roles including system-defined ones
SHOW ROLES;
-- Observe: ACCOUNTADMIN, SYSADMIN, SECURITYADMIN, USERADMIN, PUBLIC, ORGADMIN (if

-- See role hierarchy (what roles does ACCOUNTADMIN inherit?)
SHOW GRANTS TO ROLE ACCOUNTADMIN;
-- Shows: SYSADMIN and SECURITYADMIN are both granted to ACCOUNTADMIN

SHOW GRANTS TO ROLE SECURITYADMIN;
-- Shows: USERADMIN is granted to SECURITYADMIN

SHOW GRANTS TO ROLE SYSADMIN;
-- SYSADMIN does NOT have SECURITYADMIN in its grants – they are siblings, not paren

-- =====
-- PART 2: Create the recommended role hierarchy
-- =====

USE ROLE SYSADMIN;

-- Create an environment-specific database
```



```

CREATE DATABASE IF NOT EXISTS RBAC_DEMO;
USE DATABASE RBAC_DEMO;
CREATE SCHEMA IF NOT EXISTS SALES;
CREATE SCHEMA IF NOT EXISTS HR;

-- Create some tables in each schema
USE SCHEMA SALES;
CREATE TABLE IF NOT EXISTS orders (
    order_id INTEGER,
    customer VARCHAR(100),
    amount NUMBER(12,2),
    region VARCHAR(30)
);
INSERT INTO orders VALUES
    (1, 'Alice Corp', 5000, 'North'),
    (2, 'Bob LLC', 1200, 'South'),
    (3, 'Carol Inc', 8900, 'East');

USE SCHEMA HR;
CREATE TABLE IF NOT EXISTS employees (
    emp_id INTEGER,
    name VARCHAR(100),
    salary NUMBER(10,2),
    dept VARCHAR(50)
);
INSERT INTO employees VALUES
    (1, 'Alice Smith', 95000, 'Engineering'),
    (2, 'Bob Jones', 75000, 'Marketing'),
    (3, 'Carol Davis', 110000, 'Engineering');

-- =====
-- PART 3: Create access roles (fine-grained permissions)
-- =====

USE ROLE SECURITYADMIN; -- Best role for creating custom roles

-- Access roles
CREATE ROLE IF NOT EXISTS sales_read_role COMMENT = 'Can SELECT on SALES schema only';
CREATE ROLE IF NOT EXISTS hr_read_role COMMENT = 'Can SELECT on HR schema only';
CREATE ROLE IF NOT EXISTS sales_write_role COMMENT = 'Can INSERT/UPDATE on SALES tables';

-- Grant the roles to SYSADMIN (so ACCOUNTADMIN can see them – BEST PRACTICE)
GRANT ROLE sales_read_role TO ROLE SYSADMIN;
GRANT ROLE hr_read_role TO ROLE SYSADMIN;
GRANT ROLE sales_write_role TO ROLE SYSADMIN;

-- Grant object privileges to access roles
USE ROLE SYSADMIN;

-- sales_read_role needs: USAGE on DB + USAGE on schema + SELECT on tables
GRANT USAGE ON DATABASE RBAC_DEMO TO ROLE sales_read_role;
GRANT USAGE ON SCHEMA RBAC_DEMO.SALES TO ROLE sales_read_role;
GRANT SELECT ON ALL TABLES IN SCHEMA RBAC_DEMO.SALES TO ROLE sales_read_role;

-- Future tables in SALES schema also get SELECT for sales_read_role
GRANT SELECT ON FUTURE TABLES IN SCHEMA RBAC_DEMO.SALES TO ROLE sales_read_role;

```

```

-- hr_read_role
GRANT USAGE ON DATABASE RBAC_DEMO TO ROLE hr_read_role;
GRANT USAGE ON SCHEMA RBAC_DEMO.HR TO ROLE hr_read_role;
GRANT SELECT ON ALL TABLES IN SCHEMA RBAC_DEMO.HR TO ROLE hr_read_role;

-- sales_write_role (inherits sales_read_role + adds write)
GRANT ROLE sales_read_role TO ROLE sales_write_role;
GRANT INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA RBAC_DEMO.SALES TO ROLE sales_wr

-- =====
-- PART 4: Create functional roles and users
-- =====

USE ROLE SECURITYADMIN;

-- Functional roles (job titles)
CREATE ROLE IF NOT EXISTS data_analyst COMMENT = 'Data Analyst - reads SALES data';
CREATE ROLE IF NOT EXISTS data_engineer COMMENT = 'Data Engineer - reads and writes';
CREATE ROLE IF NOT EXISTS hr_manager COMMENT = 'HR Manager - reads HR data';

-- Functional roles roll up to SYSADMIN
GRANT ROLE data_analyst TO ROLE SYSADMIN;
GRANT ROLE data_engineer TO ROLE SYSADMIN;
GRANT ROLE hr_manager TO ROLE SYSADMIN;

-- Assign access roles to functional roles
GRANT ROLE sales_read_role TO ROLE data_analyst;
GRANT ROLE sales_write_role TO ROLE data_engineer; -- includes sales_read_role via
GRANT ROLE hr_read_role TO ROLE hr_manager;

-- Create users and assign functional roles
USE ROLE USERADMIN;
CREATE USER IF NOT EXISTS alice_analyst
  PASSWORD = 'Analyst@789'
  DEFAULT_ROLE = data_analyst
  DEFAULT_WAREHOUSE = learn_wh;

CREATE USER IF NOT EXISTS bob_engineer
  PASSWORD = 'Engineer@789'
  DEFAULT_ROLE = data_engineer
  DEFAULT_WAREHOUSE = learn_wh;

USE ROLE SECURITYADMIN;
GRANT ROLE data_analyst TO USER alice_analyst;
GRANT ROLE data_engineer TO USER bob_engineer;

-- =====
-- PART 5: Test privilege enforcement
-- =====

USE ROLE ACCOUNTADMIN;
USE WAREHOUSE learn_wh;

-- Test as data_analyst (should succeed for SELECT)
USE ROLE data_analyst;
SELECT * FROM RBAC_DEMO.SALES.ORDERS; -- Works!

-- Test INSERT as data_analyst (should FAIL)

```

```

INSERT INTO RBAC_DEMO.SALES.ORDERS VALUES (99, 'Test', 100, 'West');
-- Error: "Insufficient privileges to operate on table 'ORDERS'"

-- Test as data_engineer (should succeed for SELECT and INSERT)
USE ROLE data_engineer;
SELECT * FROM RBAC_DEMO.SALES.ORDERS; -- Works!
INSERT INTO RBAC_DEMO.SALES.ORDERS VALUES (99, 'Test Row', 100, 'West'); -- Works!

-- Test HR access as data_analyst (should FAIL)
USE ROLE data_analyst;
SELECT * FROM RBAC_DEMO.HR.EMPLOYEES;
-- Error: "Object 'EMPLOYEES' does not exist or not authorized"

-- =====
-- PART 6: Verify grants
-- =====

USE ROLE ACCOUNTADMIN;

-- See what's granted to a role
SHOW GRANTS TO ROLE data_analyst;
-- Shows: sales_read_role (role grant), USAGE on DB, USAGE on schema, SELECT on tabl

-- See what objects a role can access
SHOW GRANTS OF ROLE data_analyst;
-- Shows: which users and roles have data_analyst granted to them

-- See what's been granted on a specific object
SHOW GRANTS ON TABLE RBAC_DEMO.SALES.ORDERS;

-- =====
-- PART 7: Secondary Roles demo
-- =====

-- Assume alice_analyst is also granted hr_read_role for a special project:
USE ROLE SECURITYADMIN;
GRANT ROLE hr_read_role TO USER alice_analyst;

-- In alice's session:
USE ROLE data_analyst;
-- Primary role only: data_analyst can't see HR tables

USE SECONDARY ROLES ALL;
-- Now alice can access BOTH sales (via data_analyst) AND hr (via hr_read_role)!
SELECT * FROM RBAC_DEMO.HR.EMPLOYEES; -- Works with secondary roles!

USE SECONDARY ROLES NONE; -- Back to primary role only

-- Clean up
USE ROLE ACCOUNTADMIN;
DROP DATABASE IF EXISTS RBAC_DEMO;
DROP ROLE IF EXISTS sales_read_role;
DROP ROLE IF EXISTS hr_read_role;
DROP ROLE IF EXISTS sales_write_role;
DROP ROLE IF EXISTS data_analyst;
DROP ROLE IF EXISTS data_engineer;
DROP ROLE IF EXISTS hr_manager;

```

```
DROP USER IF EXISTS alice_analyst;  
DROP USER IF EXISTS bob_engineer;
```

☒ Topic 2.2 — 20 Practice Questions

Q1. In Snowflake's RBAC model, how are privileges assigned to users?

- A) Directly to users via GRANT PRIVILEGE TO USER
- B) Privileges are granted to roles; roles are granted to users
- C) Through database-level group policies similar to SQL Server
- D) Privileges inherit automatically from the user's job title

Q2. Which system role has the `MANAGE GRANTS` privilege, allowing it to grant or revoke ANY privilege on ANY object in the account?

- A) SYSADMIN
- B) ACCOUNTADMIN
- C) USERADMIN
- D) SECURITYADMIN

Q3. What does the ORGADMIN role manage?

- A) Individual databases, schemas, and tables within one account
- B) Warehouses, users, and roles across the entire Snowflake account
- C) Operations and accounts at the Organization level — multiple Snowflake accounts
- D) Encryption keys and security policies across all regions

Q4. SYSADMIN has which of the following capabilities? *(Select ALL that apply)*

- A) Create databases
- B) Create warehouses
- C) Create users
- D) Grant privileges to other roles across the account
- E) Create tables and schemas

Q5. A data engineer creates a new table in schema `ANALYTICS`. Which role automatically becomes the OWNER of that table?

- A) SYSADMIN — all tables are owned by SYSADMIN
- B) ACCOUNTADMIN — top-level role owns all new objects
- C) The role the engineer was using when they ran `CREATE TABLE`
- D) PUBLIC — all newly created objects are public by default

Q6. To query a table in Snowflake, a role needs which THREE privileges?

- A) SELECT on the table, USAGE on the database, USAGE on the schema
- B) SELECT on the table, READ on the schema, CONNECT on the database
- C) OWNERSHIP on the table, USAGE on the database, EXECUTE on the schema
- D) SELECT on the table, MODIFY on the database, INSERT on the schema

Q7. What does `GRANT SELECT ON FUTURE TABLES IN SCHEMA mydb.myschema TO ROLE analyst` do?

- A) Grants SELECT on all existing tables in the schema to the analyst role
- B) Grants SELECT automatically to the analyst role for any NEW tables created in that schema going forward
- C) Grants SELECT on the schema's future backups to the analyst role
- D) Creates a standing request that must be approved by ACCOUNTADMIN before taking effect

Q8. Which role should custom roles roll up to (be granted to) as a best practice in Snowflake?

- A) ACCOUNTADMIN
- B) SECURITYADMIN
- C) SYSADMIN
- D) PUBLIC

Q9. True or False: SYSADMIN inherits all privileges from SECURITYADMIN.

- A) True — SYSADMIN is a parent of SECURITYADMIN
- B) False — SYSADMIN and SECURITYADMIN are sibling roles. SYSADMIN does NOT inherit from SECURITYADMIN
- C) True — ACCOUNTADMIN passes all child role privileges to SYSADMIN
- D) False — No system roles inherit from any other system role

Q10. What is the purpose of `USE SECONDARY ROLES ALL`?

- A) Switches the user's active role to the next role in their role list
- B) Activates all roles granted to the user simultaneously in the current session, combining their privileges
- C) Grants all system roles (SYSADMIN, SECURITYADMIN, etc.) to the user's current session
- D) Sets the secondary authentication method for the session

Q11. Which role is AUTOMATICALLY granted to every user in a Snowflake account?

- A) SYSADMIN

- B) USERADMIN
- C) PUBLIC
- D) ACCOUNTADMIN

Q12. A role named `sales_write_role` is granted the `sales_read_role` role. A user is assigned `sales_write_role`. What access does the user have?

- A) Only the privileges defined directly on `sales_write_role` — parent roles are not inherited
- B) All privileges from both `sales_write_role` AND `sales_read_role` — role inheritance applies
- C) The user must activate `sales_read_role` separately to access its privileges
- D) The user inherits privileges from `sales_read_role` only when using `USE SECONDARY ROLES ALL`

Q13. What is DAC (Discretionary Access Control) in Snowflake?

- A) A system where all access decisions are centrally controlled by ACCOUNTADMIN with no delegation
- B) The model where each object has an owner (the role that created it) who has full rights to grant/revoke access to that object
- C) A security model where users can control their own session-level privileges
- D) An encryption model for discretionary (sensitive) data columns

Q14. USERADMIN can perform which actions? *(Select ALL that apply)*

- A) Create users (CREATE USER)
- B) Create roles (CREATE ROLE)
- C) Create databases
- D) Grant privileges on any object in the account
- E) Modify user properties (reset passwords, etc.)

Q15. A developer wants to grant privileges on a table to a role, and also allow that role to delegate (re-grant) those same privileges to other roles. Which syntax achieves this?

- A) `GRANT SELECT ON TABLE t TO ROLE r`
- B) `GRANT SELECT ON TABLE t TO ROLE r WITH GRANT OPTION`
- C) `GRANT SELECT ON TABLE t TO ROLE r ALLOW DELEGATION`
- D) `GRANT SELECT ON TABLE t TO ROLE r INHERIT`

Q16. An analyst needs to SELECT from tables in MYDB.REPORTS schema. The analyst role has been granted SELECT on all tables in that schema. However, queries still fail. What is most likely missing?

- A) The analyst needs INSERT privilege in addition to SELECT
- B) The analyst role is missing USAGE on the database MYDB and/or USAGE on the schema REPORTS
- C) The analyst needs ACCOUNTADMIN role to run SELECT queries
- D) The warehouse needs special permission to run SELECT statements

Q17. Which Snowflake account identifier format is the CURRENT preferred format?

- A) <account_locator>.loud_region_id>.loud_provider> (legacy)
- B) <org_name>-<account_name>.snowflakecomputing.com (current preferred)
- C) <account_id>@snowflake.com (new OAuth-based format)
- D) IP address + port number (direct connection)

Q18. A new table is created in SALES_DB.PUBLIC schema. The role analyst_role already has a FUTURE GRANT for SELECT on that schema. What happens?

- A) analyst_role must be re-granted SELECT on the new table manually
- B) analyst_role automatically receives SELECT on the new table — future grants apply to all new objects
- C) FUTURE GRANTS only apply when the table is explicitly added to a group
- D) The new table inherits SELECT from the schema-level privilege automatically

Q19. What does the principle of Least Privilege mean in Snowflake RBAC?

- A) Grant ACCOUNTADMIN to all users to avoid permission errors
- B) Grant users only the minimum privileges they need to perform their specific job functions
- C) Reduce the number of roles in the account to minimize complexity
- D) Use SYSADMIN for all data operations to maintain consistency

Q20. Functional roles and Access roles are a recommended RBAC design pattern. Which of the following best describes them?

- A) Functional roles own objects; Access roles query objects
- B) Functional roles represent job personas (e.g., data_analyst) that inherit Access roles representing specific object permissions (e.g., sales_read)
- C) Access roles are system-defined; functional roles are custom
- D) They are the same concept — just two names for custom roles

SUBDOMAIN 2.3 — Data Governance Features

📖 Layman's Explanation

Data Governance is about **controlling who sees what data, how data is labeled, and how data usage is tracked** — all while keeping sensitive information safe.

Imagine your company's Snowflake has a table with employee salaries and Social Security Numbers (SSNs).

- A payroll admin should see the real SSN
- A data analyst should see only `***-**-1234` (last 4 digits)
- An auditor should see a scrambled hash of the SSN

Snowflake's governance tools automate all of this:

- 📖 **Object Tags** = Sticky labels you put on tables and columns: "This column contains PII", "This is a HIPAA-regulated field"
- 📖 **Dynamic Data Masking** = Rules that say "Show the real value to payroll admins, show a hash to analysts, show stars to everyone else" — all based on the querying role
- 📖 **Row Access Policies** = Rules that say "The North region manager can only see rows where region='North'" — different users see different rows of the same table
- 📖 **Access History** = Automatic log of "Who read or wrote which column on which table at what time"
- 📖 **Data Lineage** = Visual map showing "This dashboard column came from Table A, which was loaded from Stage B, which came from Source System C"
- 📖 **Trust Center** = Snowflake's security health dashboard — "Here are your vulnerabilities, here are your compliance gaps, here is your security score"

📖 Technical Explanation

Object Tags^[20][21][^22]

What they are: Schema-level key-value metadata labels that can be applied to any Snowflake object (table, column, view, schema, database, warehouse).

How they work:

```
CREATE TAG my_schema.sensitivity_tag ALLOWED_VALUES 'PII', 'PHI', 'PUBLIC';
ALTER TABLE employees MODIFY COLUMN ssn SET TAG my_schema.sensitivity_tag = 'PII';
```

Tag propagation (Enterprise+):[^22]

- Tags applied to a source object automatically propagate to downstream objects

- Example: Tag applied to column `ssn` in table `t1` → automatically appears on views built on `t1`
- Propagation follows the lineage graph — if source object is dropped, propagation stops
- Tag values on target objects update automatically when the source tag value changes

Querying tag assignments:

```
SELECT * FROM SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES; -- All tag assignments
SELECT SYSTEM$GET_TAG('sensitivity_tag', 'employees.ssn', 'COLUMN'); -- Tag on spec
```

Edge cases:

- Tags themselves are schema-level objects — they live inside a schema
- `ALLOWED_VALUES` restricts which values can be set for a tag — optional, but best practice for consistency
- Tags can be applied to warehouses (useful for cost attribution — "this warehouse is for the Marketing team")

Dynamic Data Masking (DDM)^[23]^[18]

What it is: A Column-Level Security feature that dynamically changes what different users see when querying a column — without changing the actual stored data.

How it works:

1. Create a masking policy (a SQL function that returns masked or real data based on role)
2. Apply the policy to a column
3. When a user queries that column, the policy function runs dynamically and returns the appropriate value

Key behaviors:

- The actual data in storage is **never changed** — masking happens at query time
- Different roles see different data from the same column in the same query
- Masking policies are schema-level objects

Masking logic uses:

- `CURRENT_ROLE()` — most common; check the active role
- `IS_ROLE_IN_SESSION(role_name)` — checks if a role is anywhere in the session's role hierarchy
- `CURRENT_USER()` — mask based on specific user
- `SYSTEM$GET_TAG_ON_CURRENT_COLUMN(tag_name)` — for tag-based masking

Edge cases:

- A column can have ONLY ONE masking policy at a time
- A masked column CANNOT be used as a conditional column in another masking policy
- A column cannot be in both a masking policy AND a row access policy signature simultaneously^[24]
- Masking policies must match the data type of the column they protect (a VARCHAR masking policy cannot protect a NUMBER column)
- Snowflake evaluates the row access policy FIRST, then the masking policy^[24]

Tag-Based Masking^[20]^[23]

Combines Object Tags + Masking Policies for scalable governance:

1. Create a tag
2. Create a masking policy that reads the tag value using
`SYSTEM$GET_TAG_ON_CURRENT_COLUMN()`
3. Assign the masking policy to the tag via `ALTER TAG`
4. Any column tagged with that tag automatically gets the masking policy applied

Benefit: Instead of applying a masking policy to every column manually, you just tag the column — masking follows automatically.

```
ALTER TAG sensitivity_tag SET MASKING POLICY email_masking_policy;
-- Now any column tagged with sensitivity_tag automatically gets email_masking_policy
```

Row Access Policies (RAP)^[24]^[20]

What it is: A table/view-level policy that controls which rows a user can see based on their role, user name, or other session context.

How it works:

- Create a policy that returns TRUE (show row) or FALSE (hide row) based on session context
- Attach the policy to a table or view
- Every query against that table/view runs the policy condition against every candidate row
- Snowflake evaluates row access policy BEFORE masking policies^[24]

Common pattern: A mapping table (called an "entitlement table") maps roles/users to the values they can see.

Edge cases:

- Only ONE row access policy can be attached to a table at a time

- The same column CANNOT appear in both a row access policy signature AND a masking policy signature
- Row access policies CAN be applied to dynamic tables, external tables, and streams

Privacy Policies^[19]

A newer governance feature for controlling sensitive data:

- **Aggregation Policy:** Forces queries on a protected table to return aggregated results only — no row-level access. Ensures minimum aggregation count.
- **Projection Policy:** Prevents a column from being returned in SELECT (can be used in WHERE but not displayed).
- **Conditional Masking Policy:** Like DDM but the masking behavior is conditional on the value of another column in the same row.

Access History (Data Lineage Tracking)^{[25][26][27]}

ACCOUNT_USAGE.ACCESS_HISTORY view tracks:

- Which users accessed which columns in which tables (read access)
- Which columns were written (write access from INSERT/COPY/MERGE)
- Query IDs, timestamps, objects accessed
- Column-level lineage: how data flows from source columns to target columns in write operations

Key views in ACCOUNT_USAGE schema:^{[27][28][29]}

View	What It Contains	Retention
QUERY_HISTORY	Full SQL text, timing, warehouse, bytes scanned, partitions	365 days (1 year)
LOGIN_HISTORY	Login events, auth factors used, IP, success/failure	365 days
ACCESS_HISTORY	Column-level read and write access tracking	365 days
WAREHOUSE_METERING_HISTORY	Per-warehouse credit usage by hour	365 days
STORAGE_USAGE	Daily storage consumption (tables, stages, fail-safe)	365 days
TAG_REFERENCES	All tag assignments across the account	Active objects
COLUMNS	All column metadata	Active objects
TABLES	Table metadata including row counts	Active objects

ACCOUNT_USAGE vs INFORMATION_SCHEMA — Critical Comparison:

Feature	ACCOUNT_USAGE	INFORMATION_SCHEMA
Scope	Entire account	Current database only
Latency	Up to 45 minutes–3 hours for most views	Real-time (0 latency)
History	Up to 365 days of historical data	Current state only
Dropped objects	Included (shows dropped tables, users)	Excluded
Who can query	ACCOUNTADMIN (or GOVERNANCE_VIEWER database role)	Any role with privileges
Purpose	Historical analysis, auditing, billing	Real-time schema introspection

Critical exam trap: ACCOUNT_USAGE has latency (not real-time). INFORMATION_SCHEMA is real-time but no history.

Data Lineage in Snowsight[^30]

What it is: A visual graph in Snowsight that shows how data flows between objects — upstream sources and downstream consumers.

How to access: In Snowsight, open any table/view → click the "Lineage" tab.

Required privilege: VIEW LINEAGE on the account + appropriate privileges on the objects.

What it shows:

- Upstream: Where did this table's data come from? (source tables, stages)
- Downstream: Which views, tables, dashboards depend on this table?
- Column-level lineage: Which source columns contributed to which target columns in CTAS or INSERT...SELECT

Trust Center[^19]

What it is: Snowflake's built-in security and compliance monitoring dashboard. Automatically scans your account for:

- Security vulnerabilities (e.g., users without MFA, overprivileged roles)
- Compliance gaps (e.g., PII columns without masking policies)
- Policy violations (e.g., users sharing ACCOUNTADMIN)

Provides a **security score** and actionable recommendations.

☒ Hands-On Queries: Data Governance

```
-- =====
-- SETUP
-- =====

USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS GOVERNANCE_DEMO;
USE DATABASE GOVERNANCE_DEMO;
CREATE SCHEMA IF NOT EXISTS SENSITIVE_DATA;
USE SCHEMA SENSITIVE_DATA;
USE WAREHOUSE learn_wh;

-- Create a realistic table with PII data
CREATE OR REPLACE TABLE customer_pii (
    customer_id    INTEGER,
    full_name      VARCHAR(100),
    email          VARCHAR(200),
    phone          VARCHAR(20),
    ssn            VARCHAR(11),    -- Format: XXX-XX-XXXX
    salary         NUMBER(10,2),
    region         VARCHAR(30),
    account_tier   VARCHAR(20)    -- 'GOLD', 'SILVER', 'BRONZE'
);

INSERT INTO customer_pii VALUES
    (1001, 'Alice Johnson', 'alice@example.com', '555-0101', '123-45-6789', 95000, 'GOLD', 'US'),
    (1002, 'Bob Martinez', 'bob@example.com', '555-0102', '234-56-7890', 72000, 'SILVER', 'US'),
    (1003, 'Carol Williams', 'carol@example.com', '555-0103', '345-67-8901', 88000, 'BRONZE', 'CA'),
    (1004, 'Dave Thompson', 'dave@example.com', '555-0104', '456-78-9012', 110000, 'GOLD', 'US'),
    (1005, 'Eve Chen', 'eve@example.com', '555-0105', '567-89-0123', 65000, 'SILVER', 'CN');

-- =====
-- PART 1: Object Tags
-- =====

USE ROLE SYSADMIN;

-- Create tags for PII classification
CREATE OR REPLACE TAG sensitivity_level
    ALLOWED_VALUES 'PII', 'PHI', 'FINANCIAL', 'PUBLIC'
    COMMENT = 'Data sensitivity classification tag';

CREATE OR REPLACE TAG data_owner
    COMMENT = 'Business owner of this dataset';

-- Apply tags to columns
ALTER TABLE customer_pii MODIFY COLUMN ssn    SET TAG sensitivity_level = 'PII';
ALTER TABLE customer_pii MODIFY COLUMN salary SET TAG sensitivity_level = 'FINANCIAL';
ALTER TABLE customer_pii MODIFY COLUMN email  SET TAG sensitivity_level = 'PII';

-- Apply tag to the table itself
ALTER TABLE customer_pii SET TAG data_owner = 'Privacy Team';

-- Check what tags are on a column
SELECT SYSTEM$GET_TAG(
    'GOVERNANCE_DEMO.SENSITIVE_DATA.SENSITIVITY_LEVEL',
    'GOVERNANCE_DEMO.SENSITIVE_DATA.CUSTOMER_PII',
```

```

        'TABLE'
    ) AS table_tag;

-- Query all tag assignments for this database
SELECT
    TAG_NAME,
    TAG_VALUE,
    OBJECT_NAME,
    COLUMN_NAME,
    OBJECT_DATABASE,
    OBJECT_SCHEMA
FROM SNOWFLAKE.ACCOUNT_USAGE.TAG_REFERENCES
WHERE OBJECT_DATABASE = 'GOVERNANCE_DEMO'
ORDER BY OBJECT_NAME, COLUMN_NAME;
-- Note: ACCOUNT_USAGE has latency – may not show immediately after tagging

-- =====
-- PART 2: Dynamic Data Masking
-- =====

-- Create masking policy for SSN column
CREATE OR REPLACE MASKING POLICY ssn_mask
AS (ssn_val VARCHAR) RETURNS VARCHAR ->
CASE
    -- HR Admin sees full SSN
    WHEN IS_ROLE_IN_SESSION('SYSADMIN') THEN ssn_val
    -- Analyst sees only last 4 digits
    WHEN IS_ROLE_IN_SESSION('data_analyst') THEN
        CONCAT('***-**- ', RIGHT(ssn_val, 4))
    -- Everyone else sees fully masked
    ELSE '***-**-*****'
END;

-- Create masking policy for email
CREATE OR REPLACE MASKING POLICY email_mask
AS (email_val VARCHAR) RETURNS VARCHAR ->
CASE
    WHEN IS_ROLE_IN_SESSION('SYSADMIN') THEN email_val
    -- Show domain only
    WHEN IS_ROLE_IN_SESSION('data_analyst') THEN
        CONCAT('****@', SPLIT_PART(email_val, '@', 2))
    ELSE '****@****.***'
END;

-- Create masking policy for salary (hash for most users)
CREATE OR REPLACE MASKING POLICY salary_mask
AS (salary_val NUMBER) RETURNS NUMBER ->
CASE
    WHEN IS_ROLE_IN_SESSION('SYSADMIN') THEN salary_val
    -- Analysts see salary rounded to nearest 10K
    WHEN IS_ROLE_IN_SESSION('data_analyst') THEN
        ROUND(salary_val, -4) -- Rounds to nearest 10,000
    ELSE -1 -- Masked value
END;

-- Apply masking policies to columns

```

```

ALTER TABLE customer_pii
    MODIFY COLUMN ssn    SET MASKING POLICY ssn_mask;

ALTER TABLE customer_pii
    MODIFY COLUMN email  SET MASKING POLICY email_mask;

ALTER TABLE customer_pii
    MODIFY COLUMN salary SET MASKING POLICY salary_mask;

-- Test: As SYSADMIN (sees everything)
USE ROLE SYSADMIN;
SELECT customer_id, full_name, ssn, email, salary FROM customer_pii ORDER BY customer_id;
-- Sees: real SSN, real email, real salary

-- Test: As PUBLIC (no special role - fully masked)
USE ROLE PUBLIC;
SELECT customer_id, full_name, ssn, email, salary FROM customer_pii ORDER BY customer_id;
-- Sees: ***-**-****, ****@****.com, -1

USE ROLE SYSADMIN; -- Return to SYSADMIN

-- =====
-- PART 3: Row Access Policies
-- =====

-- Create a mapping table for region-based access control
CREATE OR REPLACE TABLE region_access_map (
    role_name  VARCHAR(100),
    region     VARCHAR(30)
);

INSERT INTO region_access_map VALUES
    ('north_manager', 'North'),
    ('south_manager', 'South'),
    ('SYSADMIN',      'North'),
    ('SYSADMIN',      'South'),
    ('SYSADMIN',      'East'),
    ('SYSADMIN',      'West');

-- Create a row access policy
CREATE OR REPLACE ROW ACCESS POLICY region_row_policy
AS (row_region VARCHAR) RETURNS BOOLEAN ->
    -- Allow if user's role is in the mapping table for this row's region
    EXISTS (
        SELECT 1
        FROM GOVERNANCE_DEMO.SENSITIVE_DATA.REGION_ACCESS_MAP m
        WHERE m.role_name = CURRENT_ROLE()
              AND m.region = row_region
    );

-- Attach row access policy to the table
ALTER TABLE customer_pii
    ADD ROW ACCESS POLICY region_row_policy ON (region);

-- Test: As SYSADMIN (can see all regions)
USE ROLE SYSADMIN;

```

```

SELECT customer_id, full_name, region FROM customer_pii ORDER BY region;
-- Shows all 5 rows (all regions in mapping table)

-- Test: Create a limited role and test (concept – needs a real test user in practice)
-- USE ROLE north_manager;
-- SELECT * FROM customer_pii;
-- Would show ONLY North region rows!

-- Verify the row access policy is attached
SHOW ROW ACCESS POLICIES IN TABLE customer_pii;

-- =====
-- PART 4: ACCOUNT_USAGE – Audit and Governance Queries
-- =====

USE ROLE ACCOUNTADMIN;

-- 1. Query history – find all queries in last hour
SELECT
    QUERY_ID,
    USER_NAME,
    ROLE_NAME,
    LEFT(QUERY_TEXT, 100)          AS query_preview,
    DATABASE_NAME,
    SCHEMA_NAME,
    QUERY_TYPE,
    EXECUTION_STATUS,
    TOTAL_ELAPSED_TIME / 1000      AS seconds,
    BYTES_SCANNED
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('hour', -1, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC
LIMIT 10;

-- 2. Access history – what columns were read recently
SELECT
    QUERY_ID,
    USER_NAME,
    QUERY_START_TIME,
    DIRECT_OBJECTS_ACCESSED        -- tables/views read
FROM SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY
WHERE QUERY_START_TIME >= DATEADD('day', -1, CURRENT_TIMESTAMP())
ORDER BY QUERY_START_TIME DESC
LIMIT 5;

-- 3. Warehouse credit usage last 7 days
SELECT
    WAREHOUSE_NAME,
    DATE(START_TIME)              AS usage_date,
    ROUND(SUM(CREDITS_USED), 3)    AS total_credits,
    ROUND(SUM(CREDITS_USED_COMPUTE), 3) AS compute_credits,
    ROUND(SUM(CREDITS_USED_CLOUD_SERVICES), 3) AS cloud_svc_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 2 DESC, 3 DESC;

```



```

-- 4. Login audit – who logged in, what auth method
SELECT
    USER_NAME,
    EVENT_TIMESTAMP,
    IS_SUCCESS,
    FIRST_AUTHENTICATION_FACTOR,
    SECOND_AUTHENTICATION_FACTOR,    -- 'DUO' or NULL if no MFA
    CLIENT_IP,
    REPORTED_CLIENT_TYPE
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE EVENT_TIMESTAMP >= DATEADD('day', -7, CURRENT_TIMESTAMP())
ORDER BY EVENT_TIMESTAMP DESC
LIMIT 20;

-- 5. Storage usage trend
SELECT
    USAGE_DATE,
    ROUND(AVERAGE_DATABASE_BYTES / 1073741824, 3)    AS db_storage_gb,
    ROUND(AVERAGE_STAGE_BYTES / 1073741824, 3)       AS stage_storage_gb,
    ROUND(AVERAGE_FAILSAFE_BYTES / 1073741824, 3)    AS failsafe_storage_gb
FROM SNOWFLAKE.ACCOUNT_USAGE.STORAGE_USAGE
WHERE USAGE_DATE >= DATEADD('day', -30, CURRENT_TIMESTAMP())
ORDER BY USAGE_DATE DESC
LIMIT 10;

-- 6. INFORMATION_SCHEMA comparison (real-time, current DB only)
USE DATABASE GOVERNANCE_DEMO;

SELECT TABLE_SCHEMA, TABLE_NAME, ROW_COUNT, BYTES
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'SENSITIVE_DATA';
-- Real-time! Includes tables created seconds ago.

-- Clean up
DROP DATABASE IF EXISTS GOVERNANCE_DEMO;

```

☒ Topic 2.3 (Data Governance) — 20 Practice Questions

Q1. What is a Snowflake Object Tag?

- A) A metadata label (key-value pair) attached to Snowflake objects — used to classify, organize, and enable governance policies on tables, columns, views, etc.
- B) A type of view that restricts access to tagged columns for non-privileged users
- C) A SQL comment added to a column definition for documentation only
- D) A special index type that speeds up queries on frequently accessed columns

Q2. What does Dynamic Data Masking (DDM) change when it masks a column?

- A) It permanently replaces the stored data with masked values
- B) It changes the column's data type to VARCHAR(masked)

- C) Nothing in storage — it transforms the returned value at query time based on the querying role, without modifying the actual stored data
- D) It encrypts the column using AES-256 and only decrypts for authorized roles

Q3. How many masking policies can be applied to a single column at one time?

- A) Unlimited — policies are stacked and evaluated in priority order
- B) Two — one for read access and one for write access
- C) One — only one masking policy can be assigned to a column at a time
- D) Three — a policy for each of: SYSADMIN, custom roles, and PUBLIC

Q4. In Dynamic Data Masking, what does `IS_ROLE_IN_SESSION('hr_admin')` check?

- A) Whether `hr_admin` is the user's default role
- B) Whether `hr_admin` is the ONLY role in the current session
- C) Whether `hr_admin` is anywhere in the current session's active role hierarchy (including secondary roles)
- D) Whether `hr_admin` has been recently activated in the session

Q5. A Row Access Policy returns `FALSE` for a specific row when queried by a particular user. What does the user see?

- A) The row with all columns fully masked
- B) The row with sensitive columns replaced by `NULL`
- C) The row is completely hidden — it is not returned in the result set at all
- D) An error message indicating the user lacks privileges

Q6. When both a Row Access Policy and a Dynamic Data Masking Policy are applied to the same table, in what order does Snowflake evaluate them?

- A) Masking Policy first, then Row Access Policy
- B) Row Access Policy first, then Masking Policy
- C) Both are evaluated simultaneously in a single scan
- D) The order depends on which policy was applied first

Q7. What is tag propagation in Snowflake?

- A) Copying tag definitions from one schema to another schema
- B) Automatically applying a tag from a source object (e.g., base table) to downstream dependent objects (e.g., views built on that table)
- C) Sending tag metadata to external governance tools via API
- D) Propagating tag privileges to child roles in the role hierarchy

Q8. The `SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY` view retains data for how long?

- A) 24 hours
- B) 7 days
- C) 90 days
- D) 365 days (1 year)

Q9. What is the KEY difference between ACCOUNT_USAGE schema views and INFORMATION_SCHEMA views?

- A) ACCOUNT_USAGE is real-time; INFORMATION_SCHEMA has up to 3-hour latency
- B) INFORMATION_SCHEMA covers the whole account; ACCOUNT_USAGE covers only the current database
- C) ACCOUNT_USAGE has up to 365 days of historical data with some latency; INFORMATION_SCHEMA is real-time but only shows current objects in the current database
- D) They are identical — INFORMATION_SCHEMA is just an alias for ACCOUNT_USAGE

Q10. Which ACCOUNT_USAGE view tracks which specific TABLE COLUMNS were read by which users?

- A) QUERY_HISTORY
- B) LOGIN_HISTORY
- C) ACCESS_HISTORY
- D) WAREHOUSE_METERING_HISTORY

Q11. What is tag-based masking in Snowflake?

- A) Masking policy that uses the column's SQL data type tag to decide masking behavior
- B) A technique combining Object Tags and Masking Policies — assigning a masking policy to a tag so any column tagged with that tag automatically gets the masking policy applied
- C) A way to tag masked columns so compliance officers can identify them
- D) Using CASE WHEN with tag values to manually apply masking in a view definition

Q12. A column is referenced in both a Row Access Policy's condition and a Masking Policy. What happens according to Snowflake's rules?

- A) Snowflake allows it — both policies evaluate the column independently
- B) This is NOT allowed — the same column cannot appear in both a masking policy signature and a row access policy signature simultaneously
- C) The masking policy takes precedence and the row access policy is ignored for that column
- D) The row access policy is evaluated first, then the masking policy evaluates the remaining visible rows

Q13. What is the Trust Center in Snowflake?

- A) A dedicated Snowflake region with enhanced physical security for regulated industries
- B) A built-in security and compliance monitoring dashboard that scans the account for vulnerabilities, policy gaps, and overprivileged roles — providing a security score and recommendations
- C) A cross-account secure data sharing feature with enhanced encryption
- D) A third-party compliance certification portal integrated with Snowflake

Q14. How does an Aggregation Policy differ from Dynamic Data Masking?

- A) Aggregation Policy masks individual column values; DDM hides entire rows
- B) DDM transforms individual column values at query time; Aggregation Policy forces queries to return only aggregated results (no row-level access) — ensuring minimum group sizes
- C) Aggregation Policy applies to numbers only; DDM applies to all data types
- D) They are the same feature with different names in different Snowflake editions

Q15. Which of the following objects does `ACCOUNT_USAGE.QUERY_HISTORY` include that `INFORMATION_SCHEMA.QUERY_HISTORY` does NOT?

- A) Recently run queries from the current session
- B) Queries run on dropped warehouses and queries from deleted users — historical data including dropped objects
- C) Queries run in the last 24 hours only
- D) Queries from all tables in the current database

Q16. What does Data Lineage in Snowflake show?

- A) The history of all user logins and their associated query counts
- B) A visual upstream/downstream map of how data flows between Snowflake objects — which tables feed which views, which views feed which dashboards
- C) The physical location of micro-partitions containing a specific table's data
- D) The replication history of databases across regions

Q17. Which privilege is required to view Data Lineage in Snowsight?

- A) `ACCOUNTADMIN` role exclusively
- B) `VIEW LINEAGE` privilege on the account (plus appropriate privileges on the objects)
- C) `SYSADMIN` role only
- D) `SELECT` on the `ACCESS_HISTORY` view in `ACCOUNT_USAGE`

Q18. A Projection Policy on a Snowflake column prevents:

- A) The column from being included in a GROUP BY clause
- B) The column value from being returned in SELECT results — the column can be used in WHERE filters but cannot be projected (displayed) to the user
- C) Users from writing data to the column via INSERT or UPDATE
- D) The column from being used in a JOIN condition

Q19. Object Tags in Snowflake can be applied to which of the following? (*Select ALL that apply*)

- A) Individual table columns
- B) Entire tables
- C) Schemas
- D) Databases
- E) Warehouses
- F) Micro-partitions within a table

Q20. Which ACCOUNT_USAGE view would you query to find the total hourly credit consumption of each warehouse over the past 30 days?

- A) QUERY_HISTORY
- B) STORAGE_USAGE
- C) WAREHOUSE_METERING_HISTORY
- D) ACCESS_HISTORY

SUBDOMAIN 2.3 (cont.) — Cost Management and Monitoring

☒ Layman's Explanation

Snowflake is a **credit-based system** — you pay for compute you use. Credits are consumed by:

- Virtual Warehouses (biggest cost driver)
- Serverless features (Automatic Clustering, Materialized Views, Snowpipe, Search Optimization)
- Cloud Services (free up to 10% of daily compute)
- Storage (charged daily, independently of compute)

The problem: Without controls, a single runaway query or a warehouse left running for days can cause a massive unexpected bill.

Snowflake's solution: Two complementary tools —

- ☒ **Resource Monitors** = Hard credit spending limits per warehouse or account. "If this warehouse uses X credits in a month, WARN me at 80%, SUSPEND new queries at 100%,"

SUSPEND ALL (cancel in-flight queries) at 110%."

- ☒ **Budgets** = Softer monthly spending projections that alert you when you're on-track to exceed a limit — uses time-series forecasting to predict end-of-month spend.

☒ Technical Explanation

Resource Monitors[^31]

What they are: Account-level or warehouse-level objects that track credit usage and take automated actions when thresholds are reached.

Key configuration parameters:

Parameter	Description
CREDIT_QUOTA	Monthly credit limit. Resets on the 1st of each calendar month (or on a custom date).
FREQUENCY	How often the quota resets: MONTHLY (default), WEEKLY, DAILY, YEARLY, NEVER
START_TIMESTAMP	When monitoring begins
NOTIFY_USERS	List of users/emails to notify when thresholds are hit

Trigger actions at threshold percentages:

Action	Description
NOTIFY	Send an alert notification. Warehouse keeps running.
SUSPEND	Prevent NEW queries from running. In-flight queries complete normally.
SUSPEND_IMMEDIATE	Stop ALL queries immediately, including in-flight ones.

Example thresholds:

```
WHEN 80 PERCENT DO NOTIFY
WHEN 100 PERCENT DO SUSPEND
WHEN 110 PERCENT DO SUSPEND_IMMEDIATE
```

Scope:

- **Account-level resource monitor:** Tracks ALL warehouses in the account
- **Warehouse-level resource monitor:** Tracks specific warehouse(s) only
- A warehouse can have only ONE resource monitor assigned at a time

Edge cases:

- Resource monitors only track WAREHOUSE credits (compute) and Cloud Services — NOT storage costs
- The SUSPEND action prevents new query submissions but does NOT cancel running queries

- SUSPEND_IMMEDIATE cancels running queries AND prevents new ones
- Quota resets to 0 at the start of each new interval regardless of how many credits were used
- Only ACCOUNTADMIN can create and manage resource monitors

Budgets^[32][33][^34]

What they are: Monthly credit spending limits with predictive alerting. Uses time-series forecasting to warn you BEFORE you exceed the limit.

Key behaviors:

- Spending limit is in Snowflake credits for one calendar month (resets on the 1st of each month)
- Sends **daily alert notifications** when current spending is forecasted to exceed the limit
- Notifications can go to: email, Amazon SNS, Azure Event Grid, Google Cloud PubSub, or webhooks (Slack, Teams, PagerDuty)

Types:

- **Account Budget:** Tracks all supported compute resources in the entire account. Only one account budget allowed.
- **Custom Budget:** User-defined group of specific Snowflake objects to track together

Application Roles for Budgets:[^33]

Role	Description
SNOWFLAKE.BUDGET_VIEWER	View budget usage data and list resources
SNOWFLAKE.BUDGET_ADMIN	Activate/deactivate, modify spending limit, configure notifications

Resource Monitor vs. Budget — Key Differences:

Feature	Resource Monitor	Budget
Primary use	Hard LIMIT — suspend warehouse when credit quota hit	Soft ALERT — forecast and notify before exceeding
Can it stop queries?	YES — SUSPEND and SUSPEND_IMMEDIATE actions	NO — alerts only, no automated actions
Scope	Warehouses only	All compute resources including serverless
Notification style	Threshold-based (80%, 100%, 110%)	Daily forecast-based alerts
Requires	ACCOUNTADMIN to create	ACCOUNTADMIN to activate account budget
Reset period	Configurable: daily, weekly, monthly, yearly	Monthly only (calendar month)

☒ Hands-On Queries: Resource Monitors and Budgets

```
-- =====
-- PART 1: Resource Monitor
-- =====

USE ROLE ACCOUNTADMIN;

-- Create a resource monitor for controlling a warehouse's credit usage
CREATE OR REPLACE RESOURCE MONITOR dev_team_monitor
  WITH CREDIT_QUOTA = 100          -- 100 credits per month
       FREQUENCY    = MONTHLY
       START_TIMESTAMP = IMMEDIATELY
       NOTIFY_USERS = ('admin@company.com')
  TRIGGERS
    ON 75 PERCENT DO NOTIFY          -- Warn at 75%
    ON 90 PERCENT DO SUSPEND         -- Block new queries at 90%
    ON 100 PERCENT DO SUSPEND_IMMEDIATE; -- Kill everything at 100%

-- Assign resource monitor to a specific warehouse
CREATE WAREHOUSE IF NOT EXISTS wh_dev_monitored
  WAREHOUSE_SIZE = 'X-SMALL'
  AUTO_SUSPEND   = 60
  AUTO_RESUME    = TRUE;

ALTER WAREHOUSE wh_dev_monitored
  SET RESOURCE_MONITOR = dev_team_monitor;

-- View the resource monitor
SHOW RESOURCE MONITORS;

-- Check resource monitor assignment on a warehouse
SHOW WAREHOUSES LIKE 'WH_DEV_MONITORED';
-- Look for: resource_monitor column

-- Create an account-level resource monitor (watches ALL warehouses)
CREATE OR REPLACE RESOURCE MONITOR account_level_monitor
  WITH CREDIT_QUOTA = 1000
       FREQUENCY    = MONTHLY
       START_TIMESTAMP = IMMEDIATELY
       NOTIFY_USERS = ('cfo@company.com', 'admin@company.com')
  TRIGGERS
    ON 50 PERCENT DO NOTIFY
    ON 80 PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND;

-- Apply to account level
ALTER ACCOUNT SET RESOURCE_MONITOR = account_level_monitor;

-- Verify
SHOW PARAMETERS LIKE 'RESOURCE_MONITOR' IN ACCOUNT;

-- =====
-- PART 2: Monitor credit usage from ACCOUNT_USAGE
-- =====

-- Daily credit consumption by warehouse (last 14 days)
```



```

SELECT
    DATE(START_TIME)                AS usage_date,
    WAREHOUSE_NAME,
    ROUND(SUM(CREDITS_USED), 3)      AS total_credits,
    ROUND(SUM(CREDITS_USED) * 3.0, 2) AS est_cost_usd,
    ROUND(SUM(CREDITS_USED_COMPUTE) -
        SUM(CREDITS_ATTRIBUTED_COMPUTE_QUERIES), 3) AS idle_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATEADD('day', -14, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 1 DESC, 3 DESC;

-- Most expensive queries in the last 24 hours
SELECT
    QUERY_ID,
    USER_NAME,
    WAREHOUSE_NAME,
    LEFT(QUERY_TEXT, 100)           AS query_preview,
    TOTAL_ELAPSED_TIME / 1000       AS seconds,
    BYTES_SCANNED,
    CREDITS_USED_CLOUD_SERVICES
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME >= DATEADD('day', -1, CURRENT_TIMESTAMP())
    AND QUERY_TYPE = 'SELECT'
ORDER BY TOTAL_ELAPSED_TIME DESC
LIMIT 10;

-- Storage cost breakdown (last 30 days)
SELECT
    USAGE_DATE,
    ROUND(AVERAGE_DATABASE_BYTES / POWER(1024,3), 2) AS db_gb,
    ROUND(AVERAGE_STAGE_BYTES / POWER(1024,3), 2)   AS stage_gb,
    ROUND(AVERAGE_FAILSAFE_BYTES / POWER(1024,3), 2) AS failsafe_gb,
    ROUND((AVERAGE_DATABASE_BYTES + AVERAGE_STAGE_BYTES +
        AVERAGE_FAILSAFE_BYTES) / POWER(1024,3), 2) AS total_gb
FROM SNOWFLAKE.ACCOUNT_USAGE.STORAGE_USAGE
WHERE USAGE_DATE >= DATEADD('day', -30, CURRENT_TIMESTAMP())
ORDER BY USAGE_DATE DESC
LIMIT 10;

-- Serverless credit consumption (Automatic Clustering, Materialized Views, etc.)
SELECT
    SERVICE_TYPE,
    DATE(START_TIME)                AS usage_date,
    ROUND(SUM(CREDITS_USED), 4) AS credits
FROM SNOWFLAKE.ACCOUNT_USAGE.SERVERLESS_TASK_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
GROUP BY 1, 2
ORDER BY 2 DESC, 3 DESC;

-- =====
-- PART 3: Budget (Snowflake Budgets)
-- =====

USE ROLE ACCOUNTADMIN;

-- Activate the account budget with a 500-credit monthly limit

```

```

-- (This uses Snowflake's built-in BUDGET class)
CALL SNOWFLAKE.CORE.BUDGET()!SET_SPENDING_LIMIT(500);
CALL SNOWFLAKE.CORE.BUDGET()!SET_EMAIL_NOTIFICATIONS(
    ENABLED          => TRUE,
    EMAIL_LIST       => ARRAY_CONSTRUCT('admin@company.com')
);
CALL SNOWFLAKE.CORE.BUDGET()!ACTIVATE();

-- View current account budget status
SELECT SNOWFLAKE.CORE.BUDGET()!GET_SPENDING_HISTORY(
    TIME_LOWER_BOUND => DATEADD('month', -1, CURRENT_DATE()),
    TIME_UPPER_BOUND => CURRENT_DATE()
);

-- Delegate budget viewing to non-admins:
GRANT APPLICATION ROLE SNOWFLAKE.BUDGET_VIEWER TO ROLE data_analyst;

-- Clean up
ALTER ACCOUNT UNSET RESOURCE_MONITOR;
DROP RESOURCE MONITOR IF EXISTS dev_team_monitor;
DROP RESOURCE MONITOR IF EXISTS account_level_monitor;
DROP WAREHOUSE IF EXISTS wh_dev_monitored;

```

☒ Topic 2.3 (Cost Management) — 20 Practice Questions

Q1. What is a Snowflake Resource Monitor?

- A) A Snowflake view that shows real-time CPU and memory usage of virtual warehouses
- B) An account-level object that tracks warehouse credit usage and can automatically notify, suspend, or kill queries when credit thresholds are reached
- C) A Snowsight dashboard tile for visualizing billing trends
- D) A monitoring agent deployed to cloud VMs to track Snowflake performance

Q2. What does the SUSPEND action in a Resource Monitor do when a threshold is reached?

- A) Cancels all currently running queries and prevents new ones
- B) Completely deletes the warehouse to prevent further credit usage
- C) Blocks new queries from being submitted but allows currently running queries to complete
- D) Sends a notification and reduces the warehouse size by one tier

Q3. What is the difference between SUSPEND and SUSPEND_IMMEDIATE in a Resource Monitor trigger?

- A) SUSPEND applies to all warehouses; SUSPEND_IMMEDIATE applies to one warehouse
- B) SUSPEND blocks new queries (in-flight complete); SUSPEND_IMMEDIATE cancels ALL queries including those currently running

- C) SUSPEND_IMMEDIATE sends an urgent notification; regular SUSPEND sends a standard notification
- D) They are identical — just different names for the same action

Q4. A Resource Monitor has CREDIT_QUOTA = 200 and FREQUENCY = MONTHLY. What happens to the credit counter on the 1st of next month?

- A) The counter continues accumulating from where it left off
- B) The counter resets to 0 — quota tracking starts fresh for the new month
- C) The quota doubles for the new month to accommodate any overage
- D) ACCOUNTADMIN must manually reset the counter each month

Q5. Which of the following credit costs does a Resource Monitor track? (*Select ALL that apply*)

- A) Virtual Warehouse compute credits
- B) Cloud Services credits attributed to warehouses
- C) Snowflake data storage costs (GB per day)
- D) Serverless credits from Automatic Clustering

Q6. A Snowflake Budget sends notifications when:

- A) A warehouse has been idle for more than 30 minutes
- B) The actual credit usage has already exceeded the spending limit
- C) Daily forecasting projects that the current spending trajectory will exceed the monthly spending limit
- D) A user runs a query that costs more than 1 credit

Q7. How does a Snowflake Budget differ from a Resource Monitor in terms of automated actions?

- A) Budgets can suspend warehouses; Resource Monitors can only send notifications
- B) Resource Monitors can suspend warehouses (automated actions); Budgets only send notifications/alerts — no automated warehouse control
- C) They both can suspend warehouses — the difference is only in alert frequency
- D) Budgets replace Resource Monitors for Enterprise edition accounts

Q8. A warehouse has a Resource Monitor assigned with CREDIT_QUOTA = 50 (MONTHLY). The account also has an account-level Resource Monitor with CREDIT_QUOTA = 500. The warehouse uses 51 credits. What happens?

- A) Only the account-level monitor triggers (warehouse-level is overridden)
- B) Only the warehouse-level monitor triggers at 100% of its 50-credit quota
- C) Both monitors trigger — each independently tracks against its own quota
- D) The warehouse is immediately permanently deleted

Q9. Which role is required to CREATE a Resource Monitor?

- A) SYSADMIN
- B) SECURITYADMIN
- C) USERADMIN
- D) ACCOUNTADMIN

Q10. The SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY view has a latency of up to how long for most columns?

- A) Real-time (0 latency)
- B) 15 minutes
- C) 3 hours (180 minutes)
- D) 24 hours

Q11. How are Snowflake Budget notifications sent? (*Select ALL that apply*)

- A) Email to specified addresses
- B) Amazon SNS topic
- C) Azure Event Grid
- D) Slack or Microsoft Teams webhooks
- E) Direct SMS text messages

Q12. A Resource Monitor's FREQUENCY = WEEKLY. The credit counter hits 100% on Wednesday. What happens to the credit counter the following Monday?

- A) It stays at 100% until manually reset by ACCOUNTADMIN
- B) It resets to 0 — the new weekly interval begins and quotas start fresh
- C) It continues accumulating — weekly frequency means 7 days after hitting 100%
- D) The warehouse is automatically dropped when the weekly limit is reached

Q13. Where would you look in Snowflake to find the total storage used by all databases, stages, and Fail-Safe over the past 30 days?

- A) SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
- B) SNOWFLAKE.ACCOUNT_USAGE.STORAGE_USAGE
- C) SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
- D) INFORMATION_SCHEMA.STORAGE_STATISTICS

Q14. A company creates a Resource Monitor at the ACCOUNT level. All warehouses in the account have NO warehouse-level Resource Monitor assigned. The account-level Resource Monitor has CREDIT_QUOTA = 1000. When this quota reaches 80% (ON 80 PERCENT DO NOTIFY), what happens?

- A) The 80% action applies only to the most credit-intensive warehouse
- B) A notification is sent; all warehouses keep running normally
- C) All warehouses are suspended simultaneously
- D) Only new warehouses created after the monitor was set are affected

Q15. What type of compute usage does a Snowflake Budget track that a Resource Monitor does NOT?

- A) Virtual Warehouse credits
- B) Cloud Services credits
- C) Serverless compute credits (Snowpipe, Automatic Clustering, Materialized Views, Search Optimization)
- D) Network transfer costs

Q16. Which ACCOUNT_USAGE view would you query to find the daily cost of Automatic Clustering and Materialized View maintenance?

- A) WAREHOUSE_METERING_HISTORY — all compute is here
- B) AUTOMATIC_CLUSTERING_HISTORY and MATERIALIZED_VIEW_REFRESH_HISTORY for serverless costs
- C) QUERY_HISTORY — filter by system maintenance queries
- D) STORAGE_USAGE — serverless costs are included here

Q17. The IDLE_COST for a warehouse can be calculated from WAREHOUSE_METERING_HISTORY as:

- A) CREDITS_USED - CREDITS_USED_CLOUD_SERVICES
- B) CREDITS_USED_COMPUTE - CREDITS_ATTRIBUTED_COMPUTE_QUERIES
- C) CREDITS_USED_COMPUTE / CREDITS_ATTRIBUTED_COMPUTE_QUERIES
- D) CREDITS_USED - CREDITS_USED_COMPUTE

Q18. A Budget's spending limit is measured in what unit?

- A) US Dollars per month
- B) Snowflake Credits per calendar month
- C) GB of data processed per week
- D) Number of queries executed per day

Q19. Which of the following are TRUE about Resource Monitors? *(Select ALL that apply)*

- A) Only ACCOUNTADMIN can create and modify resource monitors
- B) A warehouse can have only one resource monitor assigned at a time
- C) Resource monitors can track storage costs in addition to compute credits
- D) SUSPEND action blocks new queries but lets in-flight queries finish

- E) SUSPEND_IMMEDIATE stops both new and currently running queries

Q20. How does Snowflake's Hierarchical Encryption Key model protect data?

- A) A single master key encrypts all data directly — simple, fast, and secure
- B) A hierarchy of keys: Account Master Key wraps Table Master Keys, which derive File Keys that encrypt the actual data files — limiting the blast radius if any single key is compromised
- C) Each column has its own encryption key, generated at column creation time
- D) Users generate their own encryption keys that are stored in Snowflake's Cloud Services layer

☒ COMPLETE ANSWER KEY — Domain 2

Topic 2.1 — Security Principles

Q	Answer	Key Reason
1	B	Network Policies control which IP addresses can connect to Snowflake[^3]
2	B	BLOCKED_IP_LIST takes precedence over ALLOWED_IP_LIST — if in both, always blocked[^3]
3	C	Key Pair authentication (no password transmitted) is ideal for automated service accounts[^5]
4	C	Tri-Secret Secure requires Business Critical edition[^9]
5	A, B, D	Passkey (FIDO2) ✓; TOTP app ✓; Duo ✓. Hardware keys are a subcategory of passkeys. SMS is NOT supported[^6]
6	C	Revoking the CMK means Snowflake's composite master key cannot decrypt the data — access is permanently lost unless key is restored ^{[9][12]}
7	C	User-level Network Policy takes precedence over account-level for that specific user[^3]
8	B	Creating a policy has no effect until it is applied to an account or user[^3]
9	B	Two key slots enable seamless key rotation — clients migrate from key 1 to key 2 without downtime[^5]
10	C	SAML 2.0 / SSO requires a CREATE SECURITY INTEGRATION TYPE=SAML2 object to link the IdP[^4]
11	B, C, D	Auth Policies define: allowed methods ✓; MFA enrollment ✓; security integrations ✓. They don't control IP access ✗ or database access ✗[^8]
12	B	All data at rest is encrypted with AES-256 — always automatic[^11]
13	B	Snowflake automatically rotates encryption keys every 30 days[^11]
14	C	Duo Push is the default MFA method for ODBC connections[^6]
15	C	Automated scripts should use TYPE = SERVICE — requires key-pair/OAuth/PAT/WIF ^{[5][7]}

Q	Answer	Key Reason
16	A, B, D, E	TLS in transit ✓; AES-256 at rest (always) ✓; hierarchical key model ✓; Tri-Secret Secure composite key ✓. Encryption is not opt-in per table ✗ ^{[10][11]}
17	B	User-level authentication policy overrides account-level for that specific user ^[^8]
18	C	Both key slots are active simultaneously — both can authenticate — enabling zero-downtime key rotation ^[^5]
19	A, C, D, E	Account and user level ✓; user overrides account ✓; must be applied ✓; BLOCKED takes precedence ✓. Schema level ✗ ^[^3]
20	B	OAuth allows third-party tools to access Snowflake using tokens on behalf of a user — no credential sharing ^[^4]

Topic 2.2 — RBAC: Roles, Entities, and Access Control

Q	Answer	Key Reason
1	B	Privileges → Roles → Users. Never directly Privileges → Users ^{[14][15]}
2	D	SECURITYADMIN has <code>MANAGE GRANTS</code> — can grant/revoke any privilege on any object ^{[3][14]}
3	C	ORGADMIN manages the organization (multiple accounts), not a single account ^[^15]
4	A, B, E	SYSADMIN creates databases ✓, warehouses ✓, schemas/tables ✓. Does NOT create users ✗ or have <code>MANAGE GRANTS</code> ✗ ^[^3]
5	C	The role ACTIVE during <code>CREATE TABLE</code> automatically becomes OWNER of that table (DAC principle) ^[^14]
6	A	Must have: <code>USAGE</code> on database + <code>USAGE</code> on schema + <code>SELECT</code> on table — all three required ^[^3]
7	B	<code>FUTURE GRANTS</code> automatically apply <code>SELECT</code> to new tables created in that schema going forward ^[^3]
8	C	Custom roles should roll up to SYSADMIN — ensuring ACCOUNTADMIN visibility ^{[17][15]}
9	B	SYSADMIN and SECURITYADMIN are siblings. SYSADMIN does NOT inherit SECURITYADMIN ^{[14][15]}
10	B	<code>USE SECONDARY ROLES ALL</code> activates all granted roles simultaneously — combining their privileges ^[^19]
11	C	PUBLIC is automatically granted to every user in a Snowflake account ^[^14]
12	B	Role inheritance applies — <code>sales_write_role</code> grants <code>sales_read_role</code> → user gets both sets of privileges ^[^16]
13	B	DAC = every object has an owner who controls access to it (the creating role) ^[^14]
14	A, B, E	USERADMIN: <code>CREATE USER</code> ✓, <code>CREATE ROLE</code> ✓, modify user properties ✓. Cannot create databases ✗ or grant arbitrary privileges ✗ ^[^3]
15	B	<code>WITH GRANT OPTION</code> allows the grantee to re-grant the same privilege to other roles ^[^14]
16	B	Missing <code>USAGE</code> on database and/or schema — all three (DB <code>USAGE</code> + Schema <code>USAGE</code> + table <code>SELECT</code>) required ^[^3]

Q	Answer	Key Reason
17	B	<org_name>-<account_name>.snowflakecomputing.com is the current preferred format ^[19]
18	B	FUTURE GRANTS automatically apply to new objects — the new table immediately gets SELECT for analyst_role ^[3]
19	B	Least Privilege = minimum privileges needed for the job — avoid over-granting ^{[3][17]}
20	B	Functional roles = job personas that inherit Access roles = specific object permissions ^[17]

Topic 2.3 — Data Governance Features

Q	Answer	Key Reason
1	A	Object Tags are metadata key-value labels applied to Snowflake objects for classification and governance ^{[20][21]}
2	C	DDM never changes stored data — it transforms the returned value at query time only ^{[23][18]}
3	C	Only ONE masking policy can be applied to a column at a time ^[18]
4	C	IS_ROLE_IN_SESSION checks if the role appears anywhere in the session's active role hierarchy (including inherited roles) ^[23]
5	C	A row where the policy returns FALSE is completely hidden — not returned at all ^[24]
6	B	Row Access Policy is evaluated FIRST, then masking policies are applied to visible rows ^[24]
7	B	Tag propagation automatically applies tags from source objects to downstream dependent objects ^[22]
8	D	QUERY_HISTORY retains data for 365 days (1 year) ^{[25][27]}
9	C	ACCOUNT_USAGE: 365-day history, some latency, whole account, includes dropped objects. INFORMATION_SCHEMA: real-time, current DB only, current objects ^{[27][29]}
10	C	ACCESS_HISTORY tracks column-level read and write access ^{[26][27]}
11	B	Tag-based masking = assign masking policy to a tag → any column with that tag automatically gets the masking policy ^{[20][23]}
12	B	Same column CANNOT appear in both a masking policy signature and a row access policy signature simultaneously ^{[24][20]}
13	B	Trust Center is Snowflake's security health monitoring dashboard with vulnerability scanning and compliance scoring ^[19]
14	B	DDM transforms column values; Aggregation Policy forces aggregate-only results (no row-level data) ^[19]
15	B	ACCOUNT_USAGE includes data from dropped objects and deleted users — INFORMATION_SCHEMA only shows current active objects ^[27]
16	B	Data Lineage shows upstream/downstream relationships — visual map of data flow between Snowflake objects ^[30]
17	B	VIEW LINEAGE privilege on the account is required plus privileges on the objects ^[30]

Q	Answer	Key Reason
18	B	Projection Policy prevents a column from being returned in SELECT — can still be used in WHERE[^19]
19	A, B, C, D, E	Tags can be applied to columns, tables, schemas, databases, and warehouses. NOT micro-partitions ^{[20][21]}
20	C	WAREHOUSE_METERING_HISTORY tracks hourly credit consumption per warehouse ^{[28][35]}

Topic 2.3 — Cost Management Questions

Q	Answer	Key Reason
1	B	Resource Monitors track warehouse credit usage and take automated actions at thresholds[^31]
2	C	SUSPEND blocks new queries but in-flight queries complete normally[^31]
3	B	SUSPEND_IMMEDIATE cancels running queries AND blocks new ones; SUSPEND only blocks new ones[^31]
4	B	Credit counter resets to 0 at the start of the new monitoring interval[^31]
5	A, B	Resource Monitors track warehouse compute credits and cloud services credits. NOT storage costs ✖[^31]
6	C	Budgets use time-series forecasting to predict end-of-month spend and send daily notifications when on track to exceed ^{[33][34]}
7	B	Resource Monitors have automated SUSPEND/SUSPEND_IMMEDIATE actions; Budgets are notification-only ^{[33][31]}
8	C	Both monitors are independent — each tracks against its own quota. Warehouse monitor triggers at 51/50 credits (100%). Account monitor tracks cumulative account usage separately[^31]
9	D	Only ACCOUNTADMIN can create and manage Resource Monitors[^31]
10	C	WAREHOUSE_METERING_HISTORY has up to 3-hour (180-minute) latency[^28]
11	A, B, C, D	Email, SNS, Azure Event Grid, Slack/Teams webhooks all supported. SMS is NOT directly supported[^33]
12	B	Weekly frequency means counter resets on the first day of the new weekly interval[^31]
13	B	STORAGE_USAGE in ACCOUNT_USAGE tracks daily storage per database, stage, and Fail-Safe[^27]
14	B	NOTIFY action sends a notification — warehouses keep running. Only SUSPEND/SUSPEND_IMMEDIATE stops queries[^31]
15	C	Budgets track ALL compute including serverless (Snowpipe, Auto Clustering, Materialized Views); Resource Monitors track warehouses only ^{[33][34]}
16	B	AUTOMATIC_CLUSTERING_HISTORY and MATERIALIZED_VIEW_REFRESH_HISTORY are separate ACCOUNT_USAGE views for serverless credits[^27]
17	B	Idle cost = CREDITS_USED_COMPUTE - CREDITS_ATTRIBUTED_COMPUTE_QUERIES (total compute minus query-attributed compute)[^35]

Q	Answer	Key Reason
18	B	Budget spending limit is expressed in Snowflake Credits per calendar month[^33]
19	A, B, D, E	ACCOUNTADMIN only ✓; one monitor per warehouse ✓; SUSPEND allows in-flight ✓; SUSPEND_IMMEDIATE cancels all ✓. Resource Monitors do NOT track storage ✗[^31]
20	B	Hierarchical key model: Account Master Key → Table Master Keys → File Keys — limits blast radius per compromised key[^10]

References

1. [Free Practice Questions for Snowflake COF-C02 Certification](#) - Subdomain 2.1: Outline security principles. · Subdomain 2.2: Define the entities and roles that are ...
2. [Snow Pro Core Study Guide C 03 | PDF | Databases - Scribd](#) - The SnowPro Core Exam Study Guide provides an overview of the exam content, format, and preparation ...
3. [Access control best practices - Snowflake Documentation](#)
4. [Snowflake Authentication Method: How to Choose the Right One](#) - This integration supports Single Sign-On (SSO) using either OAuth or SAML 2.0, enabling MFA enforcem...
5. [Snowflake MFA & Key-Pair Authentication: How to Keep Your Data ...](#) - Two of the most important options are multi-factor authentication (MFA) for interactive users and ke...
6. [Multi-factor authentication \(MFA\) - Snowflake Documentation](#) - By default, Snowflake doesn't require MFA for users who authenticate with single sign-on (SSO). Snow...
7. [Everything You Need to Know About Snowflake's Authentication ...](#) - By April 2025, it will be mandatory for all human users on Snowflake to use multi-factor authenticat...
8. [Authentication policies | Snowflake Documentation](#) - Authentication policies provide you with control over how a client or user authenticates by allowing...
9. [Tri-Secret Secure in Snowflake](#) - Using a dual-key encryption model together with Snowflake's built-in user authentication enables thr...
10. [Understanding Encryption Key Management in Snowflake](#) - Snowflake manages data encryption keys to protect customer data. This management can occur automatic...
11. [Introducing the Snowflake Well-Architected Framework](#) - End-to-end encryption and key management: Protect data at rest and in transit using customer-control...
12. [Tri-Secret Secure with secure share area accounts in Snowflake](#) - Using a dual-key encryption model together with Snowflake's built-in user authentication enables thr...
13. [Tri-Secret Secure self-service in Snowflake](#) - Using a dual-key encryption model together with Snowflake's built-in user authentication enables thr...
14. [Overview of Access Control | Snowflake Documentation](#)
15. [Overview of Snowflake Role Based Access Control](#) - Learn various system define roles in Snowflake, their purpose and how you can create custom roles an...
16. [Snowflake Roles 101: Comprehensive Guide to Access ...](#) - Jovan Saković of Tasman Analytics provides an overview of key concepts in Snowflake access control a...

17. [Snowflake Roles and Permissions Best Practices for 2025](#) - Learn Snowflake user and role management best practices. Set up RBAC, managed schemas, and future gr...
18. [Using Dynamic Data Masking | Snowflake Documentation](#) - This topic provides instructions on how to configure and use Dynamic Data Masking in Snowflake. To l...
19. [Snowflake SnowPro Core \(COF-C03\) Full Course: Pass the Exam in 1 Video! \[7 Hours\]](#) - Master the Snowflake SnowPro Core (COF-C03) exam in this 7-hour complete masterclass! This is the on...
20. [Tag-based masking policies - Snowflake Documentation](#) - A tag-based masking policy combines the object tagging and masking policy features to allow a maskin...
21. [8 Snowflake Data Lineage Best Practices for 2026](#) - Implement object tagging on tables and columns: Enrich lineage with governance context. Enable autom...
22. [Automatic tag propagation with user-defined tags](#) - Tag propagation automatically assigns an object tag to target objects if it is applied to the source...
23. [Snowflake Masking Policies Demystified - InterWorks](#) - By combining masking policies with object tagging in Snowflake, tag-based masking policies offer a m...
24. [Understanding row access policies](#)
25. [QUERY_HISTORY view - Snowflake Documentation](#) - This Account Usage view can be used to query Snowflake query history by various dimensions (time ran...
26. [Snowflake Lineage Tag Propagation | Documentation - 2024.2 LTS](#) - The Snowflake Account Usage ACCESS_HISTORY view contains column lineage information. To appropriatel...
27. [Account Usage - Snowflake Documentation](#) - The ACCOUNT_USAGE and READER_ACCOUNT_USAGE schemas enable querying object metadata, as well as histo...
28. [WAREHOUSE_METERING_HISTORY view¶](#)
29. [Monitor query activity with Query History - Snowflake Documentation](#) - The Query History page lets you explore queries executed in your Snowflake account over the last 14 ...
30. [Data Lineage | Snowflake Documentation](#) - To view the tags on a column of the object, find the column in the Columns section. If there is a ta...
31. [Snowflake resource monitors 101: A comprehensive guide \(2026\)](#) - Improve performance and cut costs with Snowflake ...
32. [Monitor budgets | Snowflake Documentation](#) - This topic describes how to monitor budget spending and identify the budget that tracks the credit u...
33. [Monitor credit usage with budgets | Snowflake Documentation](#) - Enables a role to view budget usage data and list the resources in the budget. BUDGET_ADMIN. Enables...
34. [More Effectively Control and Limit Your Spend With Budgets](#) - Manage Snowflake spend effectively with Budgets. Learn to set spending limits, receive notifications...
35. [WAREHOUSE_METERING_HIS...](#) - This Account Usage view can be used to return the hourly credit usage for a single warehouse (or all...

